



Universidad Nacional Experimental de Guayana.
Vicerrectorado Académico.
Coordinación General de Pregrado.
Proyecto de Carrera: Ingeniería en Informática.
Unidad Curricular: Lenguajes y Compiladores.
Semestre 7 – Sección: 1 y 2.

Lenguajes y Gramáticas Formales.
Los ASTronautas
“Explorando el Universo de los Lenguajes, un Nodo a la vez”

Profesor:

Ing. Félix Márquez

Estudiantes:

C.I.: V-25.933.680. Alburquerque Sheen

C.I.: V-30.907.427. Antoima Mariangel

C.I.: V-28.475.271. García Carlos

C.I.: V-24.848.424. Varguillas Génesis.

Ciudad Guayana, Junio del 2026.

Índice.

Contenido	pág
Introducción	4
3.1. Jerarquía de Chomsky.....	5
3.1.1. Niveles de la Jerarquía	5
3.2. Alfabetos y palabras.	6
3.2.1. Definición de Palabra (o Cadena).....	7
3.2.2 Longitud de una palabra.....	7
3.2.3. Palabra vacía	7
3.2.4. Operaciones básicas con palabras.....	7
3.2.5. Clausura de Kleene	8
3.3. Lenguajes formales.....	8
3.3.1. El lenguaje de programación como lenguaje formal.....	9
3.3.2. Jerarquía de Chomsky en el contexto de la programación.....	9
3.3.3. De la especificación formal a la implementación.....	10
3.3.4. Herramientas de programación basadas en lenguajes formales	10
3.4. Gramáticas formales	11
3.4.1. Notación BNF y EBNF en programación	11
3.4.2. La Jerarquía de Chomsky para gramáticas.....	11
3.4.3. Gramáticas regulares (Tipo 3) en análisis léxico.....	12
3.4.4. Gramáticas libres de contexto (Tipo 2) en análisis sintáctico	12
3.4.5. Gramáticas sensibles al contexto (Tipo 1) y lenguajes reales.....	13
3.4.6. Gramáticas en la especificación de lenguajes.	14
3.5. Expresiones Regulares.	14
3.5.1. Operadores Fundamentales.....	14
3.5.2. Equivalencia con Autómatas Finitos	15
3.5.3. Aplicación en Análisis Léxico	15
3.5.4. Limitaciones de las Expresiones Regulares.....	16
3.5.5. Herramientas basadas en Expresiones Regulares	16
Actividad 1: Fundamentos y Jerarquía (teoría aplicada).....	17
1. Relación Gramática-Lenguaje	17
2. Jerarquía de Chomsky	18
Actividad 2: Derivación y Modelado (Caso Práctico: Genoma)	21
Actividad 3: Higiene y Optimización de Gramáticas.	26
Actividad 4: De la Expresión al Autómata (Caso Práctico: Ajedrez).....	29

1. Notación PGN (Portable Game Notation)	29
2. Definición del subconjunto simplificado	29
3. Expresión Regular	30
Conclusión.	31
Referencias.....	32

Introducción

En los temas anteriores se abordó el estudio de los lenguajes de programación desde una perspectiva práctica: se analizaron sus paradigmas, su propósito y sus estructuras morfológicas y sintácticas. Sin embargo, para diseñar, construir o incluso utilizar un compilador de manera robusta, no basta con saber cómo se ve un lenguaje; es necesario comprender cómo se genera y cómo se reconoce. Este tema introduce los fundamentos teóricos del procesamiento de lenguajes, proporcionando las herramientas formales necesarias para modelar, validar y traducir cualquier lenguaje formal.

El presente informe se centra en el estudio de los lenguajes y gramáticas formales, aplicando los conceptos teóricos a casos prácticos concretos. En primer lugar, se aborda la Jerarquía de Chomsky, los alfabetos, las palabras, los lenguajes formales, las gramáticas formales y las expresiones regulares, estableciendo el marco conceptual necesario para el análisis de lenguajes. A continuación, se presenta una gramática libre de contexto basada en el alfabeto del genoma {a, c, g, t}, utilizada para generar figuras geométricas como cuadrados, árboles y cubos, demostrando cómo una gramática puede traducirse en instrucciones de dibujo.

Posteriormente, se analizan tres patologías comunes en el diseño de gramáticas (ambigüedad, recursividad por la izquierda y factorización) y se muestran las técnicas de higiene correspondientes para transformar gramáticas problemáticas en gramáticas procesables por analizadores sintácticos. Finalmente, se aplican los conceptos de lenguajes regulares al diseño de un subconjunto del lenguaje PGN (Portable Game Notation) para movimientos de ajedrez, definiendo una gramática formal, una expresión regular y un autómata finito determinístico equivalente.

El objetivo de este trabajo es consolidar los fundamentos teóricos de los lenguajes formales y demostrar su aplicación en problemas reales de reconocimiento y procesamiento de cadenas, desde la generación de figuras geométricas hasta la validación de movimientos de ajedrez. Estos conceptos son esenciales para la construcción de compiladores, intérpretes y herramientas de procesamiento de lenguaje, y constituyen una competencia clave para el ingeniero en informática.

3.1. Jerarquía de Chomsky

La Jerarquía de Chomsky (también conocida como Jerarquía de Chomsky-Schützenberger) es una clasificación de los lenguajes formales basada en el poder generativo de las gramáticas que los definen. Propuesta por el lingüista y científico cognitivo Noam Chomsky (1956), esta jerarquía establece una estructura de anidamiento que organiza los lenguajes desde los más simples (con restricciones severas en sus reglas de producción) hasta los más complejos (sin restricciones teóricas) (Chomsky, 1956).

La relevancia de esta jerarquía en informática es fundamental, ya que define la capacidad de los diferentes tipos de autómatas y, por extensión, los límites de lo que puede ser computado o reconocido por distintos tipos de máquinas y algoritmos. Cada nivel de la jerarquía corresponde a una clase de autómata específica y a una clase de problemas computacionales (Hopcroft, Motwani & Ullman, 2007).

3.1.1. Niveles de la Jerarquía

La jerarquía se compone de cuatro niveles principales, ordenados de forma inclusiva (es decir, todo lenguaje del nivel n está contenido en el nivel $n+1$, aunque la relación inversa no es cierta).

Tipo 3: Lenguajes Regulares

- **Gramática:** Gramáticas Regulares (con reglas de la forma $A \rightarrow aB$ o $A \rightarrow a$, donde A y B son variables y a es un terminal).
- **Autómata Asociado:** Autómata Finito (Determinista o No Determinista, DFAs/NFAs).
- **Implicación Computacional:** Estos son los lenguajes computacionalmente más sencillos. Su reconocimiento requiere una cantidad de memoria fija y finita, independiente de la longitud de la entrada. Son la base de los analizadores léxicos en los compiladores y de expresiones regulares.

Tipo 2: Lenguajes Libres de Contexto (o Independientes del Contexto)

- **Gramática:** Gramáticas Libres de Contexto (con reglas de la forma $A \rightarrow \alpha$, donde A es una variable y α es una cadena de variables y terminales).
- **Autómata Asociado:** Autómata de Pila (Pushdown Automaton, PDA).
- **Implicación Computacional:** Estos lenguajes pueden describir estructuras anidadas, como los paréntesis balanceados o la sintaxis de la mayoría de los lenguajes de programación (por ejemplo, estructura if-then-else). El autómata de pila añade una pila como memoria adicional, lo que permite un contador de profundidad.

Tipo 1: Lenguajes Sensibles al Contexto (o Contextuales)

- **Gramática:** Gramáticas Sensibles al Contexto (con reglas de la forma $\alpha A \beta \rightarrow \alpha \gamma \beta$, donde $\gamma \neq \epsilon$, permitiendo que la expansión de una variable dependa de su contexto).
- **Autómata Asociado:** Autómata Linealmente Acotado (LBA).
- **Implicación Computacional:** Estos lenguajes son más complejos que los libres de contexto. Su reconocimiento consume una cantidad de memoria proporcional al tamaño de la entrada (lineal), pero puede ejecutarse durante un tiempo exponencial. Son útiles para describir lenguajes con múltiples dependencias cruzadas.

Tipo 0: Lenguajes Recursivamente Enumerables

- **Gramática:** Gramáticas Sin Restricciones (o Estructura de Frase General, con reglas de la forma $\alpha \rightarrow \beta$, sin restricciones).
- **Autómata Asociado:** Máquina de Turing (o Máquina de Turing Universal).
- **Implicación Computacional:** Representan el límite de la computabilidad. Cualquier problema que pueda ser resuelto por una computadora (es decir, que sea computable) puede expresarse en este nivel. Sin embargo, no hay garantía de que el proceso termine (problema de la parada).

La clasificación anterior es ampliamente aceptada en la literatura de teoría de autómatas y lenguajes formales (Hopcroft et al., 2007)

3.2. Alfabetos y palabras.

En la teoría de lenguajes formales, el concepto de alfabeto constituye la base fundamental sobre la que se construye cualquier lenguaje. Formalmente, se define un alfabeto (denotado comúnmente como Σ o V) como un conjunto finito y no vacío de símbolos (Hopcroft, Motwani & Ullman, 2007).

La condición de finitud es crucial desde el punto de vista computacional: una máquina abstracta o un programa de computadora solo puede reconocer un número limitado de símbolos distintos en su entrada. Algunos ejemplos típicos de alfabetos incluyen:

- $\Sigma = \{0,1\}$ (alfabeto binario, fundamental en computación)
- $\Sigma = \{a,b,c,\dots,z\}$ (alfabeto latino minúsculo)
- $\Sigma = \{d \text{ dígitos del } 0 \text{ al } 9\}$ (alfabeto decimal)
- $\Sigma = \{\text{caracteres ASCII o Unicode}\}$ (alfabeto de los lenguajes de programación)

Los símbolos que componen un alfabeto son entidades atómicas e indivisibles en el contexto de la teoría: no tienen estructura interna que el formalismo deba considerar, más allá de su identidad como elementos distintos del conjunto.

3.2.1. Definición de Palabra (o Cadena)

Una palabra (también denominada cadena o *string*) se define como una secuencia finita de símbolos pertenecientes a un alfabeto dado (Hopcroft, et al., 2007). El orden de los símbolos en la secuencia es relevante, es decir, la palabra “ab” es diferente de la palabra “ba” a menos que el alfabeto indique lo contrario por algún criterio externo.

Ejemplos de palabras sobre el alfabeto binario $\Sigma=\{0,1\}$ incluyen:

- 0 (palabra de longitud 1)
- 10110 (palabra de longitud 5)
- La palabra vacía (ver sección 3.2.3)

Notación: Es común utilizar letras minúsculas del final del alfabeto latino (como u,v,w,x,y,z) para denotar palabras. También se emplea la notación $w[i]$ para referirse al i-ésimo símbolo de la palabra w (comenzando desde 1).

3.2.2 Longitud de una palabra

La longitud de una palabra w , denotada como $|w|$, es el número de símbolos que contiene. Formalmente, si $w=a_1a_2\dots a_n$ donde cada $a_i\in\Sigma$, entonces $|w|=n$.

Propiedades importantes respecto a la longitud:

- $|\epsilon|=0$, donde ϵ es la palabra vacía.
- La longitud es un número natural (incluyendo el cero).

3.2.3. Palabra vacía

La palabra vacía (denotada comúnmente como ϵ o λ) es un concepto fundamental en la teoría de lenguajes formales. Se trata de la palabra que no contiene ningún símbolo, es decir, es la cadena de longitud cero. Cumple un papel análogo al del número cero en aritmética o al conjunto vacío en teoría de conjuntos: actúa como el elemento identidad para la operación de concatenación. Su existencia permite construir lenguajes que incluyen la posibilidad de no consumir entrada, una característica esencial en la definición de autómatas y gramáticas.

3.2.4. Operaciones básicas con palabras

A partir de las definiciones de alfabeto y palabra, se definen operaciones fundamentales que permiten construir palabras más complejas a partir de palabras más simples (Vázquez, Constable & Meloni, 2015). Estas operaciones son la base para la definición formal de lenguajes y gramáticas.

Concatenación: La concatenación de dos palabras u y v , denotada como $u \cdot v$ o simplemente uv , es la palabra formada por los símbolos de u seguidos inmediatamente por los símbolos de v . Formalmente, si $u = a_1 a_2 \dots a_m$ y $v = b_1 b_2 \dots b_n$, entonces $uv = a_1 a_2 \dots a_m b_1 b_2 \dots b_n$.

Propiedades de la concatenación:

- **Asociatividad:** $(uv)w = u(vw)$ para todas las palabras u, v, w
- **Elemento neutro:** $\epsilon w = w \epsilon = w$ para toda palabra w
- **No conmutatividad:** En general, $uv \neq vu$ (por ejemplo, sobre $\Sigma = \{a, b\}$, $ab \neq ba$)

Potencia (iteración): Para una palabra w y un número natural $n \geq 0$, se define la potencia n -ésima de w como la concatenación de w consigo misma n veces:

- $w^0 = \epsilon$
- $w^{n+1} = w^n \cdot w$

Ejemplo: sobre $\Sigma = \{a, b\}$, si $w = ab$, entonces $w^3 = (ab)^3 = ababab$.

Palabra inversa (o reverso): Dada una palabra $w = a_1 a_2 \dots a_n$, su palabra inversa o reverso, denotada como w^R o $\text{rev}(w)$, se define como $w^R = a_n a_{n-1} \dots a_1$. Es decir, es la palabra obtenida escribiendo los símbolos de w en orden inverso. Ejemplos:

- Si $w = 10110$, entonces $w^R = 01101$
- Si $w = aba$, entonces $w^R = aba$ (es un palíndromo)
- $\epsilon^R = \epsilon$

3.2.5. Clausura de Kleene

Un concepto fundamental asociado a los alfabetos es la clausura de Kleene (o estrella de Kleene), denotada como Σ^* . Se define como el conjunto de todas las palabras posibles (incluyendo la palabra vacía) que pueden formarse con símbolos del alfabeto Σ . Formalmente: $\Sigma^* = \bigcup_{n=0}^{\infty} \Sigma^n$; donde Σ^n denota el conjunto de todas las palabras de longitud n sobre Σ (con $\Sigma^0 = \{\epsilon\}$).

Observación: La clausura de Kleene Σ^* es un conjunto infinito siempre que Σ no sea vacío, porque aunque el alfabeto sea finito, la posibilidad de construir palabras de cualquier longitud genera una cantidad infinita de palabras. Por ejemplo, para $\Sigma = \{a\}$, $\Sigma^* = \{\epsilon, a, aa, aaa, aaaa, \dots\}$ es infinito numerable. También se define la clausura positiva $\Sigma^+ = \Sigma^* \setminus \{\epsilon\} = \bigcup_{n=1}^{\infty} \Sigma^n$, es decir, el conjunto de todas las palabras no vacías.

3.3. Lenguajes formales

Los lenguajes formales no son un mero constructo teórico, sino la base sobre la que se definen, implementan y procesan todos los lenguajes de programación. Todo lenguaje de programación —ya sea C, Java, Python, Rust o cualquier otro— es, ante todo, un lenguaje

formal que debe cumplir condiciones sintácticas rigurosas para ser interpretado o compilado por una máquina.

Según **Vázquez, Constable y Meloni (2015)**, "los lenguajes de programación son un ejemplo claro de lenguajes formales, ya que están definidos por reglas precisas que determinan qué cadenas de caracteres constituyen programas válidos y cuáles no".

3.3.1. El lenguaje de programación como lenguaje formal

Desde la perspectiva de la teoría de lenguajes formales aplicada a la programación:

- **Alfabeto (Σ) de un lenguaje de programación:** Conjunto de caracteres permitidos en el código fuente. Esto incluye letras (mayúsculas y minúsculas), dígitos, símbolos especiales (+, -, *, /, =, <, >, &, |, !, etc.), espacios, tabuladores y saltos de línea. En la práctica, el alfabeto suele ser el conjunto de caracteres ASCII o Unicode.
- **Palabra (cadena):** Cada programa fuente escrito por un programador es una palabra (secuencia de caracteres) sobre ese alfabeto.
- **Lenguaje (L):** El conjunto de todas las cadenas de caracteres que constituyen programas sintácticamente correctos en ese lenguaje de programación.

Ejemplo concreto (Python):

- Alfabeto: caracteres Unicode permitidos en Python 3.
- Una palabra válida: `def suma(a, b): return a + b`
- Una palabra no válida: `def suma(a, b) return a + b` (falta los dos puntos)
- El lenguaje Python es el conjunto infinito de todas las cadenas que siguen la sintaxis de Python.

3.3.2. Jerarquía de Chomsky en el contexto de la programación

La Jerarquía de Chomsky tiene aplicaciones directas y concretas en el desarrollo de compiladores e intérpretes (Hopcroft, Motwani & Ullman, 2007). La siguiente tabla resume dichas aplicaciones:

Nivel	Aplicación en programación	Ejemplo concreto
Tipo 3 (Regulares)	Análisis léxico: reconocimiento de tokens (palabras reservadas, identificadores, números, operadores, etc.)	Expresión regular para identificar un número entero en C: <code>[0-9]+</code>
Tipo 2 (Libres de contexto)	Análisis sintáctico: verificación de la estructura gramatical del programa (sentencias if-else, bucles while, declaraciones de funciones)	Gramática que define la estructura de un if en Java: <code>IfStmt $\rightarrow$ "if" "(" Expr ")" Stmt ["else" Stmt]</code>
Tipo 1 (Sensibles al contexto)	Algunas validaciones semánticas que dependen del contexto: declaración de variables antes de su uso, comprobación de tipos en ciertos lenguajes	"Una variable debe declararse antes de usarse" — esto requiere memoria de contexto (tabla de símbolos)

Tipo 0 (Recursivamente enumerables)	Teóricamente, cualquier lenguaje de programación completo (Turing-completo) pertenece aquí. En la práctica, los compiladores solo aceptan un subconjunto bien definido	Cualquier programa que eventualmente termina (o no)
---	--	---

3.3.3. De la especificación formal a la implementación

En el desarrollo de un compilador o intérprete, el lenguaje de programación se especifica formalmente mediante:

1. **Gramática libre de contexto (BNF/EBNF):** Define la sintaxis del lenguaje. Por ejemplo, la sintaxis de una sentencia while en un lenguaje tipo C se puede especificar como:

WhileStmt ::= 'while' '(' Condicion ')' Sentencia

2. **Expresiones regulares:** Definen los tokens del lenguaje (números, identificadores, palabras reservadas).

La fase de análisis léxico de un compilador se implementa típicamente como un autómata finito (reconocedor de lenguajes regulares), mientras que la fase de análisis sintáctico se implementa como un autómata de pila (reconocedor de lenguajes libres de contexto) (Hopcroft, Motwani & Ullman, 2007).

3.3.4. Herramientas de programación basadas en lenguajes formales

En la práctica profesional de la programación, existen herramientas ampliamente utilizadas que implementan directamente estos conceptos (Vázquez, Constable & Meloni, 2015):

- **Lex / Flex:** Generadores de analizadores léxicos. Aceptan especificaciones basadas en expresiones regulares (Tipo 3) y generan código en C que implementa un autómata finito.
- **Yacc / Bison:** Generadores de analizadores sintácticos. Aceptan especificaciones de gramáticas libres de contexto (Tipo 2) en notación similar a BNF y generan código en C que implementa un autómata de pila.
- **ANTLR (Another Tool for Language Recognition):** Herramienta moderna que genera analizadores en múltiples lenguajes (Java, Python, C#, etc.) a partir de gramáticas formales.

Ejemplo práctico: Un desarrollador que usa Flex para definir los tokens de un lenguaje está trabajando directamente con lenguajes regulares. Un desarrollador que escribe una gramática en ANTLR está definiendo un lenguaje libre de contexto.

3.4. Gramáticas formales

Una gramática formal es un conjunto de reglas de producción que describen cómo generar las cadenas (palabras) que pertenecen a un lenguaje formal. En el contexto de la programación, una gramática formal es la especificación matemática de la sintaxis de un lenguaje de programación (Hopcroft, Motwani & Ullman, 2007).

Formalmente, una gramática se define como una cuádrupla (V, Σ, S, P) donde:

- **V** (o **N**): Conjunto finito de símbolos no terminales (variables que representan categorías sintácticas, como "expresión", "sentencia", "declaración").
- **Σ** : Conjunto finito de símbolos terminales (los tokens del lenguaje: palabras reservadas, operadores, identificadores, números).
- **$S \in V$: Símbolo inicial** (el axioma, que representa el programa completo).
- **P**: Conjunto finito de reglas de producción (reglas de reescritura de la forma $\alpha \rightarrow \beta$).

3.4.1. Notación BNF y EBNF en programación

En la práctica de la ingeniería de software, las gramáticas formales no se presentan en notación matemática abstracta, sino mediante BNF (Backus-Naur Form) o su extensión EBNF (Extended Backus-Naur Form) (Vázquez, Constable & Meloni, 2015).

Ejemplo de BNF para una expresión aritmética simple:

```
<expresion> ::= <termino> | <expresion> "+" <termino>
<termino>  ::= <factor> | <termino> "*" <factor>
<factor>   ::= <numero> | "(" <expresion> ")"
<numero>   ::= <digito> | <digito> <numero>
<digito>   ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
```

Simbología EBNF común :

- **::=** significa "se define como"
- **|** significa "o" (alternativa)
- **{ ... }** significa "zero o más repeticiones"
- **[...]** significa "opcional (zero o una vez)"
- **(...)** se usa para agrupar

3.4.2. La Jerarquía de Chomsky para gramáticas

Según la clasificación de Noam Chomsky (1957), las gramáticas formales se organizan en cuatro niveles, siendo los dos primeros los de mayor relevancia práctica en la construcción de compiladores (Hopcroft, Motwani & Ullman, 2007). La siguiente tabla resume dicha clasificación:

Tipo	Nombre	Forma de las reglas	Aplicación en programación
Tipo 3	Gramática Regular	$A \rightarrow aB$ o $A \rightarrow a$ (donde A, B son no terminales, a es terminal)	Análisis léxico (definición de tokens: identificadores, números, palabras reservadas)
Tipo 2	Gramática Libre de Contexto (CFG)	$A \rightarrow \gamma$ (donde A es un no terminal, γ es cualquier cadena)	Análisis sintáctico (estructura de sentencias, expresiones, bloques)
Tipo 1	Gramática Sensible al Contexto (CSG)	$\alpha A \beta \rightarrow \alpha \gamma \beta$ (donde $\gamma \neq \epsilon$)	Verificaciones semánticas (declaración antes de uso, compatibilidad de tipos)
Tipo 0	Gramática Sin Restricciones	$\alpha \rightarrow \beta$ (sin restricciones)	Turing-completo; no se usa directamente en compiladores

3.4.3. Gramáticas regulares (Tipo 3) en análisis léxico

Las gramáticas regulares son las más restrictivas y corresponden exactamente a las expresiones regulares. En la práctica de la programación, se utilizan para definir los tokens del lenguaje (Vázquez, Constable & Meloni, 2015).

Ejemplo de gramática regular para números enteros en C:

```
<digito> ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
<entero> ::= <digito> | <entero> <digito>
```

Esta gramática genera cadenas como "0", "123", "4567". Es equivalente a la expresión regular $[0-9]^+$.

Herramientas de programación basadas en gramáticas regulares:

- **Lex / Flex:** Generan analizadores léxicos a partir de especificaciones de expresiones regulares.
- **JFLAP:** Herramienta educativa para diseñar y simular autómatas .

3.4.4. Gramáticas libres de contexto (Tipo 2) en análisis sintáctico

Las gramáticas libres de contexto (CFG) son el pilar del análisis sintáctico en los compiladores. La mayoría de los lenguajes de programación modernos (Java, C, Python, JavaScript) tienen una sintaxis definida mediante CFG (Hopcroft, Motwani & Ullman, 2007).

Ejemplo de CFG para una sentencia if-else en Java:

```
IfThenStatement ::= "if" "(" Expression ")" Statement
```

IfThenElseStatement ::= "if" "(" Expression ")" StatementNoShortIf "else" Statement
StatementNoShortIf ::= IfThenElseStatementNoShortIf | ... (otras sentencias)

Este ejemplo muestra cómo se resuelve el problema del *dangling else* (ambigüedad del else colgante) mediante una gramática cuidadosamente diseñada.

Herramientas de programación basadas en CFG:

- **Yacc / Bison:** Generan analizadores sintácticos (parsers) a partir de gramáticas LALR.
- **ANTLR** (Terence Parr, 2007): Genera parsers en múltiples lenguajes .
- **JFLAP:** También soporta autómatas de pila y CFG.

3.4.5. Gramáticas sensibles al contexto (Tipo 1) y lenguajes reales

Aunque la teoría indica que los lenguajes de programación deberían ser libres de contexto, en la práctica presentan características sensibles al contexto.

Ejemplo clásico en C/C++:

La declaración `a b(c);` es ambigua desde una perspectiva libre de contexto:

- Si `c` es una variable, entonces `a b(c);` es la definición de una variable `b` de tipo `a`, inicializada con `c`.
- Si `c` es un tipo, entonces `a b(c);` es la declaración de una función `b` que toma un parámetro de tipo `c` y retorna `a`.

Esta ambigüedad solo puede resolverse con información contextual (la tabla de símbolos) (Aho et al., 2008).

Otros ejemplos de sensibilidad al contexto:

- **C++ templates:** Pueden realizar cálculos en tiempo de compilación (Turing-completos) que afectan el análisis sintáctico.
- **COBOL:** La sentencia `READ` tiene una sintaxis que depende del `ACCESS MODE` del archivo (`SEQUENTIAL` vs `RANDOM`).
- **PL/I:** La palabra `IF` puede o no ser una palabra reservada según el contexto.

¿Por qué no usamos gramáticas sensibles al contexto directamente?

El reconocimiento de lenguajes sensibles al contexto es PSPACE-completo, es decir, no existen algoritmos eficientes (lineales o polinomiales) para parsearlos. Por esta razón, los compiladores utilizan una estrategia híbrida:

1. Un parser basado en CFG reconoce la estructura sintáctica básica.
2. Fases semánticas posteriores (con tabla de símbolos, verificación de tipos) manejan las restricciones contextuales.

3.4.6. Gramáticas en la especificación de lenguajes.

La especificación formal de un lenguaje de programación no se limita a una sola gramática. El estándar de C++, por ejemplo, incluye:

- Una **gramática léxica** (preprocesamiento y tokens)
- Una **gramática sintáctica** (CFG para la estructura del programa)
- **Reglas de desambiguación** (escritas en lenguaje natural)
- **Restricciones semánticas** (acceso, tipos, ámbito de nombres)

Como señala el estándar de C++ (Apéndice A): *"Esta gramática acepta un superconjunto de construcciones C++ válidas. Se deben aplicar reglas de desambiguación [...] para distinguir expresiones de declaraciones"*.

3.5. Expresiones Regulares.

Las expresiones regulares son una notación algebraica para describir patrones de cadenas sobre un alfabeto dado. Constituyen una herramienta fundamental en informática, ya que permiten especificar de manera concisa y formal los lenguajes regulares, que corresponden al nivel más bajo (Tipo 3) de la Jerarquía de Chomsky (Hopcroft, Motwani & Ullman, 2007).

En el contexto de los lenguajes formales, una expresión regular se construye recursivamente a partir de símbolos del alfabeto, utilizando tres operaciones básicas: concatenación, unión (alternancia) y clausura de Kleene (estrella).

3.5.1. Operadores Fundamentales

Sea Σ un alfabeto. Las expresiones regulares sobre Σ se definen mediante las siguientes reglas (Vázquez, Constable & Meloni, 2015):

Operador	Notación	Significado	Ejemplo
Concatenación	AB o $A \cdot B$	Una cadena de A seguida de una cadena de B	"ab" es la concatenación de "a" y "b"
Unión (Alternancia)	$A B$ o $A \cup B$	Una cadena de A o una cadena de B	"a b" reconoce "a" o "b"
Clausura de Kleene	A^*	Cero o más repeticiones de A	"a*" reconoce "", "a", "aa", "aaa", ...
Clausura Positiva	A^+	Una o más repeticiones de A	"a+" reconoce "a", "aa", "aaa", ...
Opcional	$A?$	Cero o una repetición de A	"a?" reconoce "" o "a"

La prioridad de los operadores es: clausura de Kleene (*) > concatenación > unión (|). Es decir, $ab|c^*$ se interpreta como $(ab) | (c^*)$ (Hopcroft, Motwani & Ullman, 2007).

3.5.2. Equivalencia con Autómatas Finitos

Una propiedad fundamental de las expresiones regulares es que todo lenguaje regular puede ser reconocido por un autómata finito (determinista o no determinista), y viceversa. Esta equivalencia fue demostrada por Kleene en la década de 1950 y constituye el teorema central de la teoría de lenguajes regulares (Hopcroft, Motwani & Ullman, 2007).

En la práctica, esta equivalencia permite:

- Convertir una expresión regular en un autómata finito (construcción de Thompson).
- Convertir un autómata finito en una expresión regular (método de eliminación de estados).

3.5.3. Aplicación en Análisis Léxico

La aplicación más relevante de las expresiones regulares en la construcción de compiladores es la definición de los tokens del lenguaje fuente. Un token es una unidad léxica con significado (palabra reservada, identificador, número, operador, etc.) (Aho et al., 2008).

Ejemplo de especificación de tokens mediante expresiones regulares:

Token	Expresión Regular	Descripción
Identificador	$[a-zA-Z_][a-zA-Z0-9_]^*$	Letra o guion bajo, seguida de letras, dígitos o guiones bajos
Número entero	$[0-9]^+$	Uno o más dígitos
Número real	$[0-9]^+\.[0-9]^+$	Dígitos, un punto decimal, más dígitos
Palabra reservada	$if else while for$	Palabras clave del lenguaje
Operador suma	$\backslash +$	El carácter + (escapado)
Espacio en blanco	$[\backslash t\backslash n\backslash r]^+$	Espacios, tabuladores, saltos de línea (se ignoran)

Estas expresiones regulares son procesadas por el analizador léxico (lexer), que normalmente se implementa como un autómata finito generado automáticamente por herramientas como Lex o Flex (Aho et al., 2008).

3.5.4. Limitaciones de las Expresiones Regulares

A pesar de su utilidad, las expresiones regulares tienen una limitación fundamental: no pueden reconocer lenguajes que requieren memoria adicional, como aquellos con estructuras anidadas o con dependencias entre partes distantes de la cadena.

Ejemplo clásico de lenguaje no regular: El lenguaje $\{a^n b^n \mid n \geq 0\}$ (es decir, cadenas con el mismo número de *a* que de *b*, como *ab*, *aabb*, *aaabbb*) no puede ser descrito por una expresión regular. Este lenguaje requiere contar el número de *a* para verificar que coincide con el número de *b*, lo cual excede la capacidad de memoria finita de un autómata (Hopcroft, Motwani & Ullman, 2007).

Esta limitación explica por qué el análisis sintáctico (que debe manejar estructuras anidadas como paréntesis balanceados o bloques *if-else*) requiere gramáticas libres de contexto (Tipo 2) y no simplemente expresiones regulares.

3.5.5. Herramientas basadas en Expresiones Regulares

En la práctica profesional del desarrollo de software, las expresiones regulares se utilizan en una amplia variedad de contextos (Vázquez, Constable & Meloni, 2015):

Herramienta / Contexto	Aplicación
Lex / Flex	Generación de analizadores léxicos para compiladores
Grep / Egrep	Búsqueda de patrones en archivos de texto
Lenguajes de programación (Python, JavaScript, Java, etc.)	Módulos o bibliotecas para manejo de expresiones regulares (ej. <i>re</i> en Python)
Editores de texto (VS Code, Sublime, Notepad++)	Búsqueda y reemplazo avanzado con patrones
Validación de datos	Comprobación de formatos (correos electrónicos, números de teléfono, URLs)

Ejemplo en Python:

```
import re
patron = r'[a-zA-Z_][a-zA-Z0-9_]*'
if re.match(patron, "variable_valida"):
    print("Identificador válido")
```


Actividad 1: Fundamentos y Jerarquía (teoría aplicada)

1. Relación Gramática-Lenguaje

Gramática Formal: es un conjunto finito de reglas de producción que permiten generar cadenas de símbolos siguiendo una estructura definida. Formalmente, una gramática se define como una cuádrupla $G = (N, \Sigma, P, S)$, donde (Hopcroft & Ullman, 1979):

- N es un conjunto finito de símbolos no terminales (variables)
- Σ es un conjunto finito de símbolos terminales (el alfabeto del lenguaje)
- P es un conjunto finito de reglas de producción de la forma $\alpha \rightarrow \beta$
- $S \in N$ es el símbolo inicial o axioma de la gramática

Los símbolos terminales son los elementos básicos que aparecen en las cadenas finales del lenguaje (como letras, dígitos o palabras reservadas). Los símbolos no terminales son variables auxiliares que guían el proceso de generación y deben ser reemplazadas eventualmente por terminales.

Concepto de Lenguaje Formal: Un lenguaje formal L es un subconjunto de Σ^* , donde Σ^* representa el conjunto de todas las cadenas finitas posibles formadas a partir del alfabeto Σ . En otras palabras, un lenguaje formal es un conjunto matemático de cadenas válidas que cumplen ciertas reglas estructurales (Sipser, 2012).

Relación entre Gramática y Lenguaje: La relación fundamental es que la gramática genera el lenguaje. Específicamente, el lenguaje generado por una gramática G , denotado como $L(G)$, se define como (Linz & Rodger, 2022):

$$L(G) = \{ w \in \Sigma^* \mid S \Rightarrow^* w \}$$

Es decir, el lenguaje $L(G)$ es el conjunto de todas las cadenas w de terminales que pueden ser derivadas desde el símbolo inicial S aplicando cero o más reglas de producción.

Mecanismo de Generación (Derivación): El proceso mediante el cual una gramática genera una cadena se llama **derivación**. Una derivación es una secuencia de pasos donde, en cada paso, se reemplaza un no terminal por el lado derecho de una regla de producción que lo tenga como lado izquierdo. La notación $\Rightarrow$ indica un paso de derivación, mientras que $\Rightarrow^*$ indica cero o más pasos (Hopcroft, Motwani & Ullman, 2006).

Ejemplo práctico de derivación:

Considere la gramática $G_1 = (N, \Sigma, P, S)$ donde:

- $N = \{S, NP, VP\}$
- $\Sigma = \{\text{grows, rice, wheat}\}$
- $P = \{S \rightarrow NP VP, NP \rightarrow \text{rice}, NP \rightarrow \text{wheat}, VP \rightarrow \text{grows}\}$

Para generar la cadena "rice grows", se realiza la siguiente derivación:

1. $S \Rightarrow NP VP$ (aplicando $S \rightarrow NP VP$)
2. $\Rightarrow \text{rice VP}$ (aplicando $NP \rightarrow \text{rice}$)
3. $\Rightarrow \text{rice grows}$ (aplicando $VP \rightarrow \text{grows}$)

De igual forma, para "wheat grows":

$S \Rightarrow NP VP \Rightarrow \text{wheat VP} \Rightarrow \text{wheat grows}$

El lenguaje generado por esta gramática es $L(G1) = \{\text{rice grows, wheat grows}\}$.

2. Jerarquía de Chomsky

La Jerarquía de Chomsky, propuesta por el lingüista Noam Chomsky en 1959, clasifica las gramáticas formales en cuatro tipos según la complejidad de sus reglas de producción. Esta clasificación establece una relación directa entre el poder expresivo de la gramática y la capacidad computacional necesaria para procesar el lenguaje generado (Chomsky, 1959).

Tabla Resumen de los 4 Tipos:

Tipo	Nombre	Restricción de Producciones	Lenguaje Generado	Autómata Reconocedor
Tipo 3	Regular	$A \rightarrow aB$ o $A \rightarrow a$ (con $A, B \in N, a \in \Sigma$)	Lenguaje Regular	Autómata Finito
Tipo 2	Libre de Contexto (GLC)	$A \rightarrow \gamma$ ($A \in N, \gamma \in (N \cup \Sigma)^*$)	Lenguaje Libre de Contexto	Autómata de Pila (PDA)
Tipo 1	Sensible al Contexto (GSC)	$\alpha A \beta \rightarrow \alpha \gamma \beta$ con $\gamma \neq \epsilon$ (	$\alpha A \beta$	$\leq$
Tipo 0	Sin Restricciones	$\alpha \rightarrow \beta$ (α debe contener al menos un no terminal)	Lenguaje Recursivamente Enumerable	Máquina de Turing

Fuente: Elaboración propia basada en Chomsky (1959) y Hopcroft et al. (2006)

Ejemplos en Notación BNF por Tipo

Tipo 3 (Regular) - Identificadores en Lenguajes de Programación

Las gramáticas tipo 3 (regulares) son suficientes y altamente eficientes para el análisis léxico, como la identificación de tokens (palabras clave, números, identificadores). Según Sudkamp (2006), este tipo de gramática es el fundamento teórico de los analizadores léxicos.

```
<letra> ::= "a" | "b" | "c" | "d" | "e" | "f" | "g" | "h" | "i" | "j" | "k" | "l" | "m" |  
         "n" | "ñ" | "o" | "p" | "q" | "r" | "s" | "t" | "u" | "v" | "w" | "x" | "y" | "z" |  
         "A" | "B" | "C" | "D" | "E" | "F" | "G" | "H" | "I" | "J" | "K" | "L" | "M" |  
         "N" | "O" | "P" | "Q" | "R" | "S" | "T" | "U" | "V" | "W" | "X" | "Y" | "Z"
```

```
<digito> ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
```

```
<identificador> ::= <letra> | <letra> <resto>
```

```
<resto> ::= <letra> <resto> | <digito> <resto> | ε
```

Esta gramática genera identificadores como "x", "variable1", "contador2", etc.

Tipo 2 (Libre de Contexto) - Expresiones Aritméticas

Las gramáticas tipo 2 son fundamentales para definir la sintaxis de los lenguajes de programación, permitiendo estructuras anidadas como paréntesis balanceados o bloques de código. Hopcroft et al. (2006) señalan que la mayoría de los lenguajes de programación tienen una sintaxis que puede ser descrita por una GLC.

```
<E> ::= <E> "+" <T> | <T>
```

```
<T> ::= <T> "*" <F> | <F>
```

```
<F> ::= "(" <E> ")" | "a" | "b" | "c"
```

Esta gramática genera expresiones aritméticas como "a + b * c" o "(a + b) * c", respetando la jerarquía de operadores.

Tipo 1 (Sensible al Contexto) - Lenguaje $a^n b^n c^n$

Las gramáticas tipo 1 son más potentes que las libres de contexto y pueden generar lenguajes que requieren "recordar" contextos. Un ejemplo clásico es el lenguaje $\{ a^n b^n c^n \mid n \geq 1 \}$, que NO puede ser generado por una gramática tipo 2 (Kozen, 1997).

$\langle S \rangle ::= "a" \langle S \rangle "B" "C" \mid "a" "B" "C"$

$\langle C \rangle "B" ::= "B" \langle C \rangle$

$"a" "B" ::= "a" "b"$

$"b" "B" ::= "b" "b"$

$"b" "C" ::= "b" "c"$

$"c" "C" ::= "c" "c"$

Las reglas contextuales como $\langle C \rangle "B" \rightarrow "B" \langle C \rangle$ permiten reorganizar los símbolos para generar cadenas con igual número de a's, b's y c's (Sudkamp, 2006).

Tipo 0 (Sin Restricciones) - Gramática Universal

Las gramáticas tipo 0 no tienen restricciones en sus producciones. El lenguaje generado por este tipo de gramática es equivalente al que puede reconocer una Máquina de Turing, lo que representa el máximo poder computacional. Sipser (2012) explica que estos lenguajes son los más generales que se pueden definir computacionalmente.

$\langle S \rangle ::= "a" \langle A \rangle "c"$

$\langle A \rangle ::= \langle B \rangle "b" \mid "b"$

$"b" \langle B \rangle ::= \langle B \rangle "b"$

$\langle B \rangle "c" ::= "b" \langle A \rangle "c"$

Esta gramática ejemplifica cómo las reglas pueden tener múltiples símbolos tanto en el lado izquierdo como en el derecho, sin restricciones de longitud o contexto.

Chomsky, N. (1959). [Sobre ciertas propiedades formales de las gramáticas]. Information and Control, 2(2), 137-167.

Actividad 2: Derivación y Modelado (Caso Práctico: Genoma)

Gramática Libre de Contexto (GLC)

Definición formal: Una GLC es una cuádrupla $G=(V,\Sigma,R,S)$ donde (Hopcroft, Motwani & Ullman, 2007):

- V : conjunto finito de variables (no terminales)
- Σ : alfabeto terminal (disjunto de V)
- R : conjunto finito de reglas de producción de la forma $A \rightarrow \alpha$ con $A \in V$, $\alpha \in (V \cup \Sigma)^*$
- $S \in V$: símbolo inicial

Derivación

Una derivación es una secuencia de reemplazos de no terminales usando las reglas de producción. Se denota $\alpha \Rightarrow^* \beta$ si β se obtiene desde α en cero o más pasos (Hopcroft, Motwani & Ullman, 2007).

Lenguaje generado

$$L(G) = \{w \in \Sigma^* \mid S \Rightarrow^* w\}$$

Aplicación a dibujo (Turtle Graphics)

Se define una función de interpretación $I: \Sigma^* \rightarrow \text{Imagen}$ que asigna a cada cadena terminal una figura geométrica. Cada símbolo terminal es un comando para un "tortuga virtual" (Vázquez, Constable & Meloni, 2015).

Gramática para dibujar (Caso Genoma)

La idea central es construir una Gramática Libre de Contexto (GLC) cuyo alfabeto terminal $\Sigma = \{a, c, g, t\}$ coincide con las bases nitrogenadas del ADN (adenina, citosina, guanina, timina), pero reinterpretadas como instrucciones para una tortuga gráfica (turtle graphics), de forma análoga a cómo funcionan los Sistemas-L (L-Systems) usados para modelar estructuras biológicas (plantas, fractales, ramificaciones celulares) (Prusinkiewicz & Lindenmayer, 1990).

Cada símbolo del genoma se traduce en una acción de dibujo:

Símbolo	Acción de dibujo (semántica)	Equivalente turtle
a	Avanzar dibujando un segmento de línea unitario	F
c	Girar 90° (sentido horario), sin dibujar	+
g	Abrir rama: guardar posición/orientación actuales en una pila y desviar el trazo	[
t	Cerrar rama: recuperar de la pila la posición/orientación guardadas	]

Convención: El primer a después de t baja automáticamente el lápiz.

No terminales:

- D : dibujo principal
- S : secuencia
- F : figura básica
- R : repetición
- B : bifurcación (ramificación)

Definición formal de la gramática G

$G = (N, \Sigma, P, S)$

- $N = \{S, \text{CUADRADO}, \text{LADO}, \text{ARBOL}, \text{RAMA}, \text{HOJA}, \text{CUBO}, \text{CARA}, \text{DIAGONAL}\}$
- $\Sigma = \{a, c, g, t\}$
- S = símbolo inicial

Producciones (P):

- (1) $S \rightarrow \text{CUADRADO}$
- (2) $\text{CUADRADO} \rightarrow \text{LADO LADO LADO LADO}$
- (3) $\text{LADO} \rightarrow a c$
- (4) $S \rightarrow \text{ARBOL}$
- (5) $\text{ARBOL} \rightarrow a \text{ RAMA}$
- (6) $\text{RAMA} \rightarrow \text{HOJA}$
- (7) $\text{RAMA} \rightarrow g \text{ ARBOL } t c \text{ ARBOL}$
- (8) $\text{RAMA} \rightarrow g \text{ ARBOL } t g \text{ ARBOL } t c \text{ ARBOL}$
- (9) $\text{HOJA} \rightarrow a$
- (10) $S \rightarrow \text{CUBO}$
- (11) $\text{CUBO} \rightarrow \text{CARA } g \text{ CARA } t \text{ DIAGONAL}$
- (12) $\text{CARA} \rightarrow \text{CUADRADO}$
- (13) $\text{DIAGONAL} \rightarrow a a a a$

Derivaciones: Definición formal y ejemplos

Derivación 1 — Cuadrado

$S \Rightarrow (1) \text{ CUADRADO}$
 $\Rightarrow (2) \text{ LADO LADO LADO LADO}$
 $\Rightarrow (3) a c \text{ LADO LADO LADO}$
 $\Rightarrow (3) a c a c \text{ LADO LADO}$
 $\Rightarrow (3) a c a c a c \text{ LADO}$
 $\Rightarrow (3) a c a c a c a c$

Cadena final: **acacacac**

Interpretación: el patrón "ac" se repite 4 veces. Cada "a" traza un lado de longitud unitaria y cada "c" gira 90° . Como $4 \times 90^\circ = 360^\circ$, el lápiz regresa al punto y orientación iniciales, cerrando un cuadrado.

Derivación 2 — Árbol con una rama lateral y hojas

S $\Rightarrow$ (4) ARBOL
 $\Rightarrow$ (5) a RAMA
 $\Rightarrow$ (7) a g ARBOL t c ARBOL
 $\Rightarrow$ (5) a g a RAMA t c ARBOL
 $\Rightarrow$ (6) a g a HOJA t c ARBOL
 $\Rightarrow$ (9) a g a a t c ARBOL
 $\Rightarrow$ (5) a g a a t c a RAMA
 $\Rightarrow$ (6) a g a a t c a HOJA
 $\Rightarrow$ (9) a g a a t c a a

Cadena final: **agaatcaa**

Interpretación: "a" dibuja el tronco; "g" abre una rama lateral, que se dibuja y termina en una hoja ("aa"); "t" cierra la rama y regresa al tronco; "c" gira para retomar la dirección del tronco, que continúa y finaliza también en una hoja ("aa"). Resultado: árbol con una rama y dos hojas.

Derivación 3 — Árbol con ramificación anidada (varias ramas)

S $\Rightarrow$ (4) ARBOL
 $\Rightarrow$ (5) a RAMA
 $\Rightarrow$ (7) a g ARBOL t c ARBOL
 $\Rightarrow$ (5) a g a RAMA t c ARBOL
 $\Rightarrow$ (7) a g a g ARBOL t c ARBOL t c ARBOL
 $\Rightarrow$ (5) a g a g a RAMA t c ARBOL t c ARBOL
 $\Rightarrow$ (6) a g a g a HOJA t c ARBOL t c ARBOL
 $\Rightarrow$ (9) a g a g a a t c ARBOL t c ARBOL
 $\Rightarrow$ (5) a g a g a a t c a RAMA t c ARBOL
 $\Rightarrow$ (6) a g a g a a t c a HOJA t c ARBOL
 $\Rightarrow$ (9) a g a g a a t c a a t c ARBOL
 $\Rightarrow$ (5) a g a g a a t c a a t c a RAMA
 $\Rightarrow$ (6) a g a g a a t c a a t c a HOJA
 $\Rightarrow$ (9) a g a g a a t c a a t c a a

Cadena final: **agagaatcaatcaa**

Interpretación: el tronco se ramifica ("g") en una rama que vuelve a ramificarse internamente (segunda "g") antes de terminar en una hoja; al cerrar esa subrama, la rama original continúa

hasta su propia hoja; al cerrar la rama principal, el tronco original termina en su propia hoja. Resultado: árbol con tres niveles de ramificación y tres hojas — análogo a la recursividad de un Sistema-L.

Derivación 4 — Árbol con doble ramificación simétrica

S ⇒ (4) ARBOL
 ⇒ (5) a RAMA
 ⇒ (8) a g ARBOL t g ARBOL t c ARBOL
 ⇒ (5) a g a RAMA t g ARBOL t c ARBOL
 ⇒ (6) a g a HOJA t g ARBOL t c ARBOL
 ⇒ (9) a g a a t g ARBOL t c ARBOL
 ⇒ (5) a g a a t g a RAMA t c ARBOL
 ⇒ (6) a g a a t g a HOJA t c ARBOL
 ⇒ (9) a g a a t g a a t c ARBOL
 ⇒ (5) a g a a t g a a t c a RAMA
 ⇒ (6) a g a a t g a a t c a HOJA
 ⇒ (9) a g a a t g a a t c a a

Cadena final: **agaatgaatcaa**

Interpretación: el tronco genera una primera rama lateral ("g...aa...t") con hoja; vuelve al punto del tronco y genera una segunda rama lateral simétrica ("g...aa...t") con hoja; finalmente "c" retoma la dirección del tronco, que concluye en su propia hoja. Resultado: árbol con dos ramas simétricas y tres hojas en total.

Derivación 5 — Cubo (proyección isométrica)

S ⇒ (10) CUBO
 ⇒ (11) CARA g CARA t DIAGONAL
 ⇒ (12) CUADRADO g CARA t DIAGONAL
 ⇒ (2) LADO LADO LADO LADO g CARA t DIAGONAL
 ⇒ (3×4) a c a c a c a c g CARA t DIAGONAL
 ⇒ (12) a c a c a c a c g CUADRADO t DIAGONAL
 ⇒ (2) a c a c a c a c g LADO LADO LADO LADO t DIAGONAL
 ⇒ (3×4) a c a c a c a c g a c a c a c a c t DIAGONAL
 ⇒ (13) a c a c a c a c g a c a c a c a c t a a a a

Cadena final: **acacacacgacacacacttaaa** → ordenado: **acacacac · g · acacacac · t · aaaa**

Interpretación: primero se dibuja un cuadrado completo ("acacacac") que representa la cara frontal del cubo; "g" desplaza el punto de referencia en diagonal (profundidad) sin perder la posición original; allí se dibuja un segundo cuadrado idéntico ("acacacac"), correspondiente a la cara posterior; "t" recupera el punto de referencia de la cara frontal; y los cuatro símbolos "a"

finales ("aaaa") representan las cuatro aristas diagonales que conectan cada vértice de la cara frontal con su correspondiente vértice en la cara posterior, completando la proyección isométrica del cubo.

Modelado: De la derivación a la representación gráfica

El modelado consiste en:

1. **Definir la GLC** (V, Σ, R, S)
2. **Generar una cadena** mediante derivación (puede ser manual o con un generador sintáctico)
3. **Interpretar** la cadena como comandos de tortuga
4. **Renderizar** la figura resultante

Algoritmo de interpretación (pseudocódigo)

```
estado = (x=0, y=0, orientación=0° , lápiz_abajo=True)
for símbolo in cadena:
    if símbolo == 'a':
        if lápiz_abajo: dibujar_línea(estado, avanzar(1))
        else: mover_sin_dibujar(estado, avanzar(1))
        lápiz_abajo = True # tras t, el primer a baja lápiz
    elif símbolo == 'c':
        orientación -= 90°
    elif símbolo == 'g':
        orientación += 90°
    elif símbolo == 't':
        lápiz_abajo = False
```

Limitaciones de la GLC para modelado de dibujo

Limitación	Explicación	Posible solución
Sin repetición numérica	No se puede expresar "a ⁵ " compactamente	Añadir reglas con índices: $A \rightarrow anA \rightarrow an$ o usar gramática con parámetros
Sin variables de estado	La posición/orientación no son parte de la gramática	Usar gramática dependiente del contexto o atributos
Sin estructuras de control complejas (bucles, condicionales)	Solo recursión lineal y bifurcación simple	Extender a gramática de gráficos (ej. sistemas de reescritura de grafos)
Ambigüedad	Una misma figura puede tener múltiples derivaciones	No afecta al dibujo, pero sí al análisis sintáctico

Actividad 3: Higiene y Optimización de Gramáticas.

Las gramáticas mal diseñadas pueden causar errores en los compiladores, como bucles infinitos, ambigüedad en la interpretación de las cadenas o ineficiencia en el análisis sintáctico. A continuación se presentan tres patologías comunes y sus respectivas soluciones (Hopcroft, Motwani & Ullman, 2007; Aho et al., 2008).

a) Gramática Ambigua.

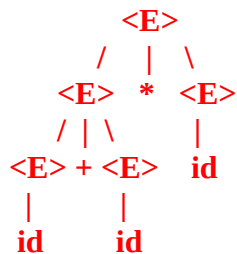
Una gramática es ambigua si existe al menos una cadena que puede ser derivada mediante dos árboles de derivación distintos. Esto es indeseable en los lenguajes de programación porque un programa no debería tener más de un significado posible.

Ejemplo de gramática ambigua para expresiones aritméticas:

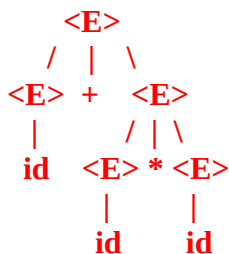
$\langle E \rangle ::= \langle E \rangle + \langle E \rangle \mid \langle E \rangle * \langle E \rangle \mid \text{id}$

Cadena ambigua: $\text{id} + \text{id} * \text{id}$

Árbol de derivación 1 (interpretación: $(\text{id} + \text{id}) * \text{id}$):



Árbol de derivación 2 (interpretación: $\text{id} + (\text{id} * \text{id})$):



Solución: Reescribir la gramática introduciendo niveles de precedencia:

$$\begin{aligned} \langle E \rangle &::= \langle E \rangle + \langle T \rangle \mid \langle T \rangle \\ \langle T \rangle &::= \langle T \rangle * \langle F \rangle \mid \langle F \rangle \\ \langle F \rangle &::= \text{id} \mid "(" \langle E \rangle ")" \end{aligned}$$

Esta gramática elimina la ambigüedad porque fuerza la precedencia de los operadores: la multiplicación (*) tiene mayor prioridad que la suma (+) (Aho et al., 2008).

b) Recursividad por la Izquierda.

Un problema común en los analizadores sintácticos descendentes (top-down) es la recursividad por la izquierda. Una gramática es recursiva por la izquierda si contiene una regla de la forma $A \rightarrow A \alpha$. Esto provoca bucles infinitos en los analizadores recursivos.

Ejemplo de gramática con recursividad por la izquierda:

$$\begin{aligned} \langle E \rangle &::= \langle E \rangle + \langle T \rangle \mid \langle T \rangle \\ \langle T \rangle &::= \text{id} \end{aligned}$$

Cadena problemática: id + id + id

Algoritmo de eliminación de recursividad por la izquierda:

Dada una regla de la forma $A \rightarrow A \alpha \mid \beta$ (donde β no comienza con A), se transforma en:

$$\begin{aligned} A &\rightarrow \beta A' \\ A' &\rightarrow \alpha A' \mid \epsilon \end{aligned}$$

Aplicación paso a paso a la gramática anterior:

1. Identificar la regla problemática: $\langle E \rangle \rightarrow \langle E \rangle + \langle T \rangle \mid \langle T \rangle$

Aquí $A = \langle E \rangle$, $\alpha = + \langle T \rangle$, $\beta = \langle T \rangle$

2. Aplicar la transformación:

$$\begin{aligned} \langle E \rangle &\rightarrow \langle T \rangle \langle E' \rangle \\ \langle E' \rangle &\rightarrow + \langle T \rangle \langle E' \rangle \mid \epsilon \end{aligned}$$

3. La gramática resultante (sin recursividad por la izquierda) es:

$$\begin{aligned} \langle E \rangle &\rightarrow \langle T \rangle \langle E' \rangle \\ \langle E' \rangle &\rightarrow + \langle T \rangle \langle E' \rangle \mid \epsilon \end{aligned}$$

$\langle T \rangle \rightarrow id$

Esta gramática puede ser procesada sin problemas por un analizador descendente (Hopcroft, Motwani & Ullman, 2007).

c) Factorización por la Izquierda: es una técnica que se aplica cuando dos o más reglas de un mismo no terminal comparten un prefijo común. Esto causa ambigüedad temporal en el analizador, que no sabe qué regla elegir hasta leer más tokens.

Ejemplo de gramática que requiere factorización:

$\langle A \rangle ::= id = \langle E \rangle \mid id (\langle E \rangle)$

Ambas reglas comienzan con el prefijo común id . El analizador no puede decidir qué regla aplicar hasta leer el siguiente token ($=$ o $()$).

Algoritmo de factorización por la izquierda:

Dadas las reglas de la forma $A \rightarrow \alpha \beta_1 \mid \alpha \beta_2 \mid \dots \mid \alpha \beta_n$, se transforman en:

$A \rightarrow \alpha A'$

$A' \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$

Aplicación a la gramática anterior:

1. Identificar el prefijo común: $\alpha = id$
2. Identificar los sufijos: $\beta_1 = = \langle E \rangle$ y $\beta_2 = (\langle E \rangle)$
3. Aplicar la transformación.

Gramática optimizada resultante:

$\langle A \rangle \rightarrow id \langle A' \rangle$

$\langle A' \rangle \rightarrow = \langle E \rangle \mid (\langle E \rangle)$

Esta gramática factorizada permite al analizador leer el token id y luego decidir qué camino tomar basándose en el siguiente token (si es $=$ o $()$) (Aho et al., 2008).

Actividad 4: De la Expresión al Autómata (Caso Práctico: Ajedrez).

1. Notación PGN (Portable Game Notation)

PGN es un estándar para representar partidas de ajedrez mediante texto. Para este proyecto no se modela el lenguaje completo, sino un subconjunto orientado al reconocimiento léxico. Desde la teoría de autómatas, las cadenas PGN pueden analizarse como palabras pertenecientes a un lenguaje formal definido sobre un alfabeto finito.

2. Definición del subconjunto simplificado

Peón simple: e4, d5, a3
Peón con captura: exd5, cxb2
Caballo simple y captura: Nf3, Nxe5
Alfil simple y captura: Bc4, Bxh7
Torre simple y captura: Re1, Rxa7
Dama simple y captura: Qh5, Qxe5
Rey simple y captura: Ke2, Kxf3
Enroque corto: O-O
Jaque opcional: Qh5+, exd5+

Alfabeto Σ :

{K,Q,R,B,N,a,b,c,d,e,f,g,h,1,2,3,4,5,6,7,8,x,+,O,-}

Gramática Formal:

Sea $G=(V,\Sigma,P,S)$

Producciones principales:

$S \rightarrow \text{MOV} \mid \text{MOV JAQUE}$
 $\text{MOV} \rightarrow \text{PEON} \mid \text{PIEZA} \mid \text{ENROQUE}$
 $\text{PEON} \rightarrow \text{COL FILA} \mid \text{COL x COL FILA}$
 $\text{PIEZA} \rightarrow \text{PIEZAID COL FILA} \mid \text{PIEZAID x COL FILA}$
 $\text{PIEZAID} \rightarrow \text{K} \mid \text{Q} \mid \text{R} \mid \text{B} \mid \text{N}$
 $\text{ENROQUE} \rightarrow \text{O-O}$
 $\text{JAQUE} \rightarrow +$
 $\text{COL} \rightarrow \text{a} \mid \text{b} \mid \text{c} \mid \text{d} \mid \text{e} \mid \text{f} \mid \text{g} \mid \text{h}$
 $\text{FILA} \rightarrow 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8$

3. Expresión Regular

La expresión regular diseñada para reconocer el lenguaje es:

$$([a-h][1-8]|[a-h]x[a-h][1-8]|([KQRBN][a-h][1-8]|([KQRBN]x[a-h][1-8]|O-O)\+)?$$

Esta expresión reconoce movimientos simples, capturas, enroque corto y la marca opcional de jaque.

Diseño del AFD: El AFD se construyó a partir de la expresión regular. Desde el estado inicial se distinguen tres ramas principales: movimientos de peón, movimientos de pieza y enroque. Los estados de aceptación representan cadenas válidas y existe una transición opcional para el símbolo '+' de jaque.

Estados: $Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7, q_8, q_9, q_F, q_{10}\}$

Estado inicial: q_0

Estados de aceptación: $\{q_F, q_{10}\}$

Tabla de Transiciones (Resumen)

$q_0 \xrightarrow{\text{--COL--}} q_1$
 $q_0 \xrightarrow{\text{--PIEZA--}} q_2$
 $q_0 \xrightarrow{\text{--O--}} q_8$
 $q_1 \xrightarrow{\text{--FILA--}} q_F$
 $q_1 \xrightarrow{\text{--x--}} q_3$
 $q_3 \xrightarrow{\text{--COL--}} q_6$
 $q_6 \xrightarrow{\text{--FILA--}} q_F$
 $q_2 \xrightarrow{\text{--COL--}} q_4$
 $q_4 \xrightarrow{\text{--FILA--}} q_F$
 $q_2 \xrightarrow{\text{--x--}} q_5$
 $q_5 \xrightarrow{\text{--COL--}} q_7$
 $q_7 \xrightarrow{\text{--FILA--}} q_F$
 $q_8 \xrightarrow{\text{----}} q_9$
 $q_9 \xrightarrow{\text{--O--}} q_F$
 $q_F \xrightarrow{\text{--+++}} q_{10}$

Ejemplos de Cadenas

Aceptadas:

e4, exd5, Nf3, Nxe5, Bc4, Rxa7, Qxe5, Ke2, O-O, Qh5+

Rechazadas:

z9, e0, Pxe5, Nxxe5, O--O, Q++, +

Conclusión.

El estudio de los lenguajes y gramáticas formales constituye un pilar fundamental en la formación de un ingeniero en informática, ya que proporciona las bases teóricas para comprender cómo se generan, validan y procesan los lenguajes de programación. A lo largo de este trabajo, se han abordado los conceptos centrales de la Jerarquía de Chomsky, los alfabetos y palabras, los lenguajes formales, las gramáticas formales y las expresiones regulares, estableciendo un marco conceptual sólido que conecta la teoría con la práctica.

En la primera parte del informe, se analizó la relación entre gramáticas y lenguajes formales, comprendiendo cómo una gramática, a través de sus reglas de producción, puede generar todas las cadenas válidas de un lenguaje. Esta relación es fundamental para el diseño de compiladores, donde la gramática define la sintaxis del lenguaje fuente y guía el proceso de análisis sintáctico. La Jerarquía de Chomsky, por su parte, permitió clasificar las gramáticas según su poder expresivo y comprender qué tipo de autómatas es necesario para reconocer cada nivel.

La segunda parte del trabajo abordó la aplicación práctica de estos conceptos mediante el modelado de figuras geométricas a través de una Gramática Libre de Contexto basada en el alfabeto del genoma {a, c, g, t}. Esta actividad demostró cómo una gramática puede generar cadenas que, interpretadas como comandos de dibujo, producen estructuras complejas como árboles y cubos, evidenciando la relación entre la teoría de lenguajes y la generación de patrones visuales mediante sistemas-L.

En la tercera parte, se estudiaron las patologías más comunes en el diseño de gramáticas: la ambigüedad, la recursividad por la izquierda y la falta de factorización. Cada una de estas patologías puede causar errores en los compiladores, como bucles infinitos o ambigüedad en la interpretación de las cadenas. Se presentaron las técnicas de higiene correspondientes (reescritura de gramáticas, eliminación de recursividad y factorización por la izquierda) que permiten transformar gramáticas problemáticas en gramáticas procesables eficientemente por analizadores sintácticos.

Finalmente, en la cuarta actividad, se aplicaron los conceptos de lenguajes regulares al diseño de un subconjunto del lenguaje PGN (Portable Game Notation) para movimientos de ajedrez. Se definió una gramática formal, se construyó una expresión regular y se diseñó un Autómata Finito Determinístico (AFD) equivalente, demostrando que los movimientos básicos de ajedrez pueden ser reconocidos por autómatas finitos, lo que confirma que este subconjunto pertenece al Tipo 3 de la Jerarquía de Chomsky.

En conjunto, este trabajo ha permitido consolidar los fundamentos teóricos de los lenguajes formales y su aplicación en problemas concretos de reconocimiento y procesamiento de cadenas, desde la generación de figuras geométricas hasta la validación de movimientos de ajedrez. La comprensión de estos conceptos es esencial para la construcción de compiladores, intérpretes y herramientas de procesamiento de lenguaje.

Referencias.

- Chomsky, N. (1956). Three models for the description of language. IRE Transactions on Information Theory, 2(3), 113-124.
- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). Introducción a la teoría de autómatas, lenguajes y computación (3ª ed.). Pearson Educación.
- Vázquez, J., Constable, L., & Meloni, B. (2015). Lenguajes formales y teoría de autómatas. Alfaomega.
- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2008). Compiladores: principios, técnicas y herramientas (2ª ed.). Pearson Educación.
- Hopcroft, J. E., & Ullman, J. D. (1979). Introduction to automata theory, languages, and computation (1st ed.). Addison-Wesley.
- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). Introduction to automata theory, languages, and computation (3rd ed.). Pearson Education.
- Linz, P., & Rodger, S. H. (2022). An introduction to formal languages and automata (7th ed.). Jones & Bartlett Learning.
- Sipser, M. (2012). Introduction to the theory of computation (3rd ed.). Cengage Learning.
- Kozen, D. C. (1997). Automata and computability. Springer.
- Sudkamp, T. A. (2006). Languages and machines: An introduction to the theory of computer science (3rd ed.). Pearson Addison-Wesley.
- Prusinkiewicz, P., & Lindenmayer, A. (1990). The algorithmic beauty of plants. Springer-Verlag.